

Task 07 - Part 02: Question Answers

C# Fundamentals - class vs struct & Relationships Between Classes

Q1. What is the difference between class and struct in C#?

The core difference is that a **class** is a **reference type** while a **struct** is a **value type**, and everything else largely follows from that:

- **Memory:** A class instance is allocated on the heap, and the variable holds a reference (address) to it. A struct instance is typically allocated on the stack (or inline within its containing object), holding its data directly.
- **Copy behavior:** Assigning or passing a class copies the reference, so both variables point to the same object - changes through one are visible through the other. Assigning or passing a struct copies the entire value, producing two fully independent copies.
- **Inheritance:** A class supports inheritance (from one base class) and polymorphism. A struct cannot inherit from another struct or class (though it can implement interfaces), and cannot be inherited from.
- **Null:** A class variable can be null (it's a reference). A struct cannot be null unless wrapped as a nullable value type (e.g., MyStruct?).
- **Typical use:** Classes are generally preferred for larger, more complex objects with identity and behavior that should be shared/mutated across references. Structs are best for small, simple, immutable-ish data (like a Point or a Color) where value semantics and avoiding heap allocations are beneficial.

Q2. If inheritance is a relation between classes, clarify other relations between classes.

Inheritance (an "is-a" relationship) is only one of several ways classes can relate to each other in object-oriented design. The other common relationships are:

- **Association** - a general, loose relationship where one class uses or is connected to another, without one owning the other's lifetime (e.g., a Teacher class is associated with a Student class - each can exist independently of the other).
- **Aggregation** - a "has-a" (whole-part) relationship where one class contains another, but the contained object can exist independently of the container (e.g., a Department "has" Employees, but an Employee can still exist if the Department is deleted).
- **Composition** - a stronger "has-a" relationship where the contained object's lifetime is tied to the container - it cannot meaningfully exist without it (e.g., a House "owns" its Rooms; if the House is destroyed, the Rooms cease to exist too).
- **Dependency** - a weaker, often temporary relationship where one class simply uses another, typically as a method parameter, local variable, or return type, without holding a lasting reference to it (e.g., a PrintService that takes a Document as a parameter to a Print(Document doc) method).

- **Realization / Implementation** - the relationship between a class and an interface it implements (a class "realizes" the contract defined by an interface), which is distinct from inheriting a base class's implementation.

In short, while inheritance models an "is-a" relationship (a Child *is* a Parent), these other relationships mostly model different flavors of "has-a" or "uses-a" connections between otherwise unrelated classes.